Musztáng SE

Teszt Dokumentáció

Ganz Ábrahám Két Tanítási Nyelvű Technikum

Nagy Eszter Dóra
Zsitva Dániel Béla

Tartalom

1. Tesztek 2
1.1 Tesztelési terv 2
1.1.1 Tesztelési célok 2
1.1.2 Tesztelési típusok 2
1.2 Unit Tesztek 3
1.2.1 Tesztprojekt felépítése 3
1.2.2 TestDbContextHelper 3
1.2.3 PasswordService tesztek 4
1.2.4 VersenyService tesztek 5
1.2.5 JwtService tesztek 6
1.2.6 AuthController tesztek 6
1.2.7 Tesztek futtatása 7
1.3 Egységtesztek 8
1.4 Integrációs tesztek 8
1.5 Manuális funkcionális tesztek 8
1.6 Tesztelési környezet 8
1.7 Tesztesetek 9
1.7.1 Bejelentkezés / Jelszó beállítás 9
1.7.2 Edzői felület 10
1.8 Swagger API tesztek 11
1.8.1 Tesztesetek 12

1. Tesztek

1.1 Tesztelési terv

A tesztelési terv célja a Musztáng SE webalkalmazás funkcióinak átfogó ellenőrzése, a hibák feltárása és a felhasználói élmény optimalizálása. A tesztelés kiterjed a backend üzleti logikára, a frontend működésére, a szerepkör alapú jogosultságkezelésre, valamint a reszponzív megjelenítésre.

1.1.1 Tesztelési célok

- Az alkalmazás alapvető funkcióinak ellenőrzése (bejelentkezés, adatkezelés, jogosultságkezelés)
- A felhasználói élmény értékelése különböző eszközökön (asztali számítógép, tablet, mobiltelefon)
- A szerepkör alapú hozzáférés ellenőrzése — tag, edző, elnök és adminisztrátor funkciók elkülönítése
- A biztonsági és adatvédelmi szempontok felülvizsgálata (JWT token kezelés, jelszó titkosítás)
- Az API végpontok helyes működésének ellenőrzése
- A felhasználói felület reszponzivitásának tesztelése

1.1.2 Tesztelési típusok

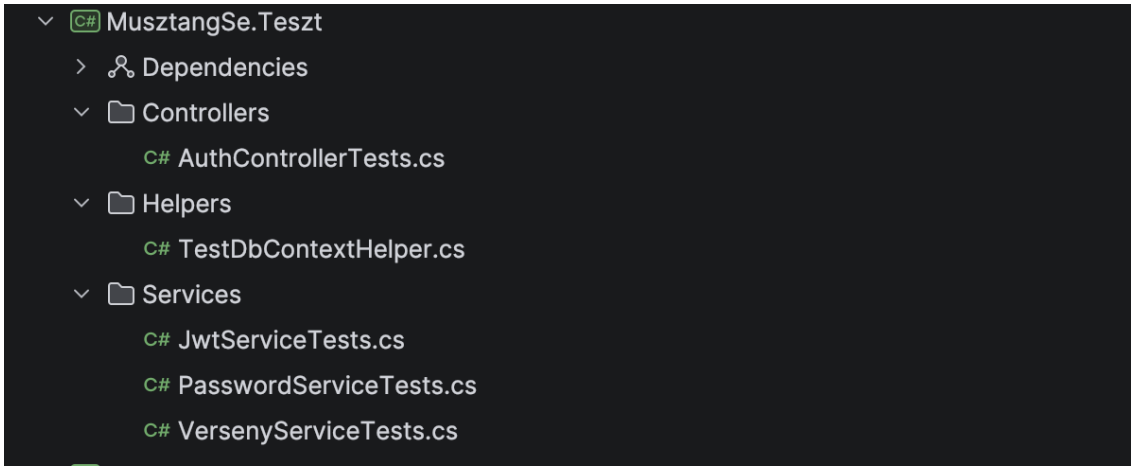
- **Unit tesztek** — Az alkalmazás egyes komponenseinek izolált tesztelése, amelyek az üzleti logika helyességét ellenőrzik valódi adatbázis kapcsolat nélkül. Az unit tesztek xUnit keretrendszerrel készültek és InMemory adatbázist használnak.
- **Manuális funkcionális tesztek** — A felhasználói felület és az API végpontok kézi tesztelése, amelyek a teljes alkalmazás működését ellenőrzik valódi környezetben. Az API végpontok tesztelése Swagger felületen keresztül történt.
- **Reszponzivitási tesztek** — A felhasználói felület megjelenésének ellenőrzése különböző képernyőméreteken, a hamburger menü és a mobilos elrendezés működésének tesztelése.
- **Biztonsági tesztek** — A JWT token alapú autentikáció és a szerepkör alapú jogosultságkezelés ellenőrzése, annak tesztelése hogy illetéktelen felhasználók nem férhetnek hozzá védett tartalmakhoz.

1.2 Unit Tesztek

A projekt backend logikájának ellenőrzésére unit teszteket készítettünk, amelyek az alkalmazás kritikus üzleti logikáját tesztelik. A tesztek xUnit keretrendszerrel készültek, és a Visual Studio beépített Test Explorer eszközével futtathatók.

1.2.1 Tesztprojekt felépítése

A tesztprojekt (`MusztangSE.Teszt`) a solution részeként külön projektként lett létrehozva, az



alábbi mappaszerkezettel:

A tesztprojekt az alábbi NuGet csomagokat használja:

Csomag	Szerepe
xunit	Tesztelési keretrendszer
Moq	Mock objektumok létrehozása
Microsoft.EntityFrameworkCore.InMemory	Valódi adatbázis helyett memóriában futó adatbázis tesztekhez

1.2.2 TestDbContextHelper

A `Helpers/TestDbContextHelper.cs` fájl egy segédosztályt tartalmaz, amely minden teszthez egy friss, üres InMemory adatbázis kontextust hoz létre. Az InMemory adatbázis használatának előnye hogy a tesztek nem igényelnek valódi adatbázis kapcsolatot, így gyorsan és egymástól függetlenül futtathatók.

1.2.3 PasswordService tesztek

A `Services/PasswordServiceTests.cs` fájl a jelszókezelési logikát teszteli. A tesztelt esetek:

Teszt neve	Leírás	Elvárt eredmény
Hash_NemAdjaVissza_AzEredetiJelszot	A hash nem egyezik meg az eredeti jelszóval	A hash ≠ eredeti jelszó
Hash_NemNull	A hash nem null értékű	A visszaadott érték nem null
Verify_HelytesJelszoEseten_IgazatAd	Helyes jelszóval a verify igaz értéket ad	true
Verify_HelytElenJelszoEseten_HamisatAd	Helytelen jelszóval a verify hamis értéket ad	false
KetKulonbozoHash_UgyanazonJelszobol_NemEgyenlo	Ugyanabból a jelszóból két különböző hash jön létre (salt miatt)	hash1 ≠ hash2

1.2.4 VersenyService tesztek

A `Services/VersenyServiceTests.cs` fájl a versenyek lekérdezési logikáját teszteli InMemory adatbázis segítségével. A tesztelt esetek:

Teszt neve	Leírás	Elvárt eredmény
GetUpcomingAsync_CsakJovobeliVersenyeketAdVissza	A lekérdezés csak jövőbeli versenyeket ad vissza	1 db verseny
GetUpcomingAsync_DatumSzerintNovekvoenRendez	A versenyek dátum szerint növekvő sorrendben jelennek meg	Az első verseny a legkorábbi
GetUpcomingAsync_UresListat_AdVissza_HaNincsKozelgoVerseny	Ha nincs közelgő verseny, üres lista érkezik vissza	Üres lista
GetAllAsync_MindenVersenyeketAdVissza	Az összes verseny visszaadásra kerül függetlenül a dátumtól	2 db verseny

1.2.5 JwtService tesztek

A `Services/JwtServiceTests.cs` fájl a JWT token generálási logikát teszteli. A konfigurációt egy InMemory `IConfiguration` példány biztosítja, így nem szükséges valódi `appsettings.json` fájl. A tesztelt esetek:

Teszt neve	Leírás	Elvárt eredmény
GenerateToken_NemNull	A generált token nem null és nem üres	Nem null, nem üres string
GenerateToken_ErvenyesJwtFormatot_AllitElo	A token érvényes JWT formátumú	A `JwtSecurityTokenHandler` el tudja olvasni
GenerateToken_TartalmazzaARoleClaim-et	A token tartalmazza a felhasználó szerepkörét	A `role` claim értéke egyezik a szerepkörrel

1.2.6 AuthController tesztek

A `Controllers/AuthControllerTests.cs` fájl a bejelentkezési logikát teszteli. A teszt előkészítése során egy InMemory adatbázisba kerül egy tesztfelhasználó, és egy valódi `JwtService` példány kerül példányosításra. A tesztelt esetek:

Teszt neve	Leírás	Elvárt eredmény
Login_HelytesAdatokkal_OkEredmenyt_AdVissza	A generált Helyes azonosítóval és jelszóval sikeres bejelentkezésnem null és nem üres	200 OK
Login_RosszJelszoVal_UnauthorizedEredmenyt_AdVissza	Helytelen jelszóval sikertelen bejelentkezés	401 Unauthorized
Login_NemLetezoFelhasznaloval_UnauthorizedEredmenyt_AdVissza	Nem létező azonosítóval sikertelen bejelentkezés	401 Unauthorized

1.2.7 Tesztek futtatása

A tesztek kétféleképpen futtathatók:

Visual Studio Test Explorer:

- Test → Run All Tests menüpont
- Az eredmények zöld pipával (sikeres) vagy piros X-szel (sikertelen) jelennek meg

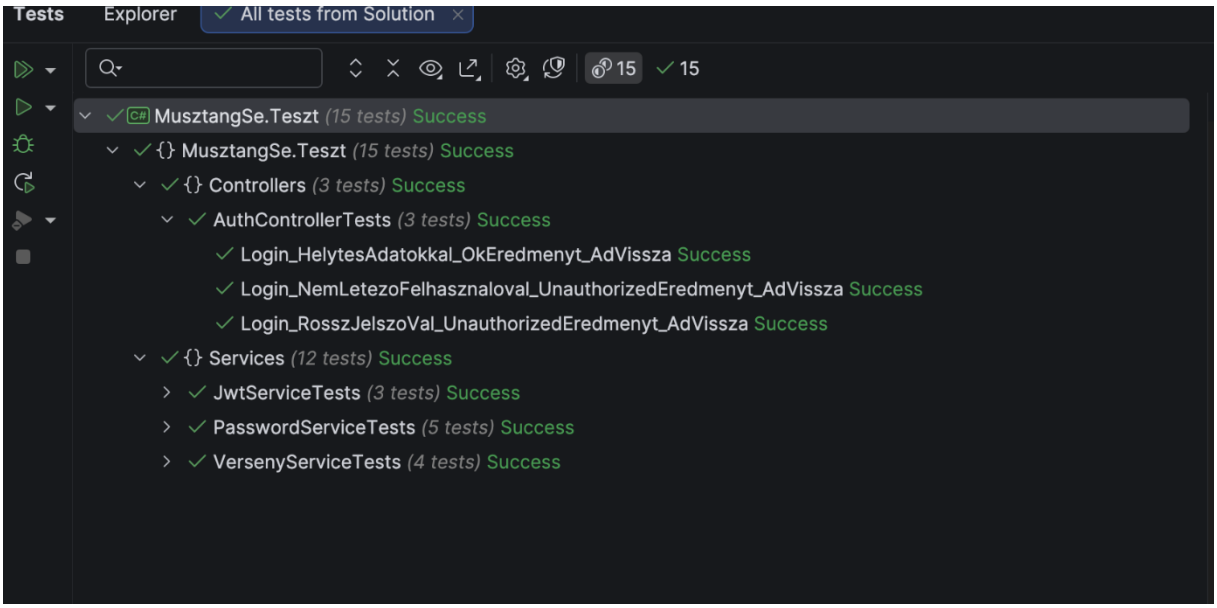
Terminálból:

cd MusztangSE.Tests

dotnet test

Sikeres futás esetén az alábbi kimenet jelenik meg:

Passed! - Failed: 0, Passed: 15, Skipped: 0, Total: 15



1.3 Egységtesztek

- **Backend működés:** Az adatbázis-műveletek ellenőrzése, például az adatok mentése és lekérdezése.
- **Validációk**: Annak vizsgálata, hogy az űrlapok megfelelően kezelik-e a hibás vagy hiányzó adatokat.

1.4 Integrációs tesztek

- Frontend és backend együttműködés: Az adatküldés és -megjelenítés folyamatának zavartalan működésének ellenőrzése.
- Jogosultságkezelés: Annak tesztelése, hogy a különböző szerepkörök a megfelelő hozzáférési jogosultságokkal rendelkeznek.

1.5 Manuális funkcionális tesztek

- Felhasználói felület és API tesztelése: Az alkalmazás teljes működésének ellenőrzése valós környezetben, kézi teszteléssel.
- API végpontok vizsgálata: A backend szolgáltatások tesztelése Swagger felületen keresztül.

1.6 Tesztelési környezet

- Böngészők: Chrome, Firefox, Edge, Safari
- Eszközök: Asztali számítógép, tablet, mobiltelefon
- Operációs rendszerek: Windows, macOS, iOS, Android

1.7 Tesztesetek

1.7.1 Bejelentkezés / Jelszó beállítás

Tesztelési cél	Lépések	Elvárt eredmény
Sikeres jelszó beállítás	1. Jelszóbeállítás felület megnyitása 2. Összes mező kitöltése 3. Megerősítés gomb megnyomása	Sikeres jelszóbeállítás visszajelzés. Visszautasítás a bejelentkezés oldalra.
Hibás azonosító	Nem létező azonosító megadása	Hibaüzenet: „Felhasználó nem található.”
Nem megegyező jelszó	Két különböző jelszó megadása	Hibaüzenet: „A két jelszó nem egyezik.”
Sikeres belépés	Bejelentkezés egy helyes azonosító + jelszó kombinációval	Sikeres bejelentkezés visszajelzés
Sikertelen belépés	Bejelentkezés egy hibás azonosító + jelszó kombinációval vagy rossz szerepkör kiválasztásával.	Hibaüzenet: „Hibás azonosító vagy jelszó” vagy „A kiválasztott szerepkör nem egyezik a fiókod típusával.”

1.7.2 Edzői felület

1.7.2.1 Tag hozzáadása

Tesztelési cél	Lépések	Elvárt eredmény
Sikeres tag hozzáadás	1. Tagok oldalon új tag felvétele gomb 2. Összes mező kitöltése 3. Mentés gomb megnyomása	Sikeres tagfelvétel visszajelzés.
Ki nem töltött mezők	Csillagozott mező(k) üresen hagyása	Hibaüzenet: „*Üres mező (k) név* kötelező!”
Nem generált azonosító	Minden mező kitöltése viszont „Azonosító generálás” gomb meg nem nyomása	Hibaüzenet: „Kérlek generálj azonosítót a mentés előtt!

1.7.2.2 Eredmény hozzáadása

Tesztelési cél	Lépések	Elvárt eredmény
Sikeres eredmény hozzáadás	1. Eredmények oldalon „új eredmény” gomb 2. Összes mező kitöltése 3. Mentés gomb megnyomása	Sikeres eredmény felvétel visszajelzés.
Ki nem töltött mezők	Csillagozott mező(k) üresen hagyása	Hibaüzenet: „*Üres mező név* kötelező!”

1.8 Swagger API tesztek

A backend API végpontjainak tesztelése a Swagger felületen keresztül történt, amely a fejlesztői környezetben a `https://musztangse-api-gghga9fnd3eqetcd.westeurope-01.azurewebsites.net/swagger/index.html` címen érhető el.

Authorize gomb használata

Mivel a legtöbb végpont JWT token alapú autentikációt igényel, a tesztelés előtt a következő lépéseket kellett elvégezni:

1. Az `api/auth/login` végponton bejelentkeztem a megfelelő szerepkörű felhasználóval
2. A visszakapott tokent kimásoltam
3. A Swagger felület jobb felső sarkában található **Authorize** gombra kattintottam
4. A megjelenő ablakban a `Bearer {token}` formátumban beírtam a tokent
5. Az **Authorize** gombra kattintva a token érvénybe lépett és az összes védett végpont elérhetővé vált

1.8.1 Tesztesetek

Végpont	Metódus	Szerepkör	Bemenet	Várt eredmény	Kapott eredmény
api/auth/login	POST	-	Helyes azonosító és jelszó	200 OK + token	200 OK + token
api/auth/login	POST	-	Helytelen jelszó	401 Unauthorized	401 Unauthorized
api/auth/login	POST	-	Nem létező azonosító	401 Unauthorized	401 Unauthorized
api/auth/setpassword	POST	-	Helyes adatok	200 OK	200 OK
api/shared/versenyek/kozelgo	GET	tag/edzo/admin	-	200 OK + lista	200 OK + lista
api/shared/versenyek/ev/{ev}	GET	tag/edzo/admin	Évszám	200 OK + lista	200 OK + lista
api/coach/tagok	GET	edzo	-	200 OK + tagok listája	200 OK + tagok listája
api/coach/tagok	POST	edzo	Új tag adatai	200 OK	200 OK
api/coach/tagok/{id}	PUT	edzo	Módosított adatok	200 OK	200 OK
api/coach/tagok/{id}	DELETE	edzo	Tag ID	204 No Content	204 No Content
api/coach/csapatok/sajat	GET	edzo	-	200 OK + csapatok	200 OK + csapatok
api/coach/csapatok	POST	edzo	Új csapat adatai	200 OK	200 OK
api/coach/csapatok/{id}/archival	PATCH	edzo	Csapat ID	200 OK	200 OK

api/coach/csapatok/tag-hozzaadas	POST	edzo	TagId, CsapatId	200 OK	200 OK
api/coach/csapatok/tag-eltavolitas/{csapatId}/{tagId}	DELETE	edzo	ID-k	200 OK	200 OK
api/coach/eredmenyek	GET	edzo	-	200 OK + lista	200 OK + lista
api/coach/eredmenyek	POST	edzo	Eredmény adatai	200 OK	200 OK
api/coach/eredmenyek/{id}	PUT	edzo	Módosított adatok	200 OK	200 OK
api/coach/eredmenyek/{id}	DELETE	edzo	Eredmény ID	200 OK	200 OK
api/tag/me/adataim	GET	tag	-	200 OK + adatok	200 OK + adatok
api/tag/me/adataim	PUT	tag	Módosított adatok	200 OK	200 OK
api/admin/dashboard	GET	admin	-	200 OK + statisztikák	200 OK + statisztikák
api/admin/felhasznalok	GET	admin	-	200 OK + lista	200 OK + lista
api/admin/felhasznalok/edzo	POST	admin/edzo	Új edző adatai	200 OK	200 OK
api/admin/felhasznalok/{id}	DELETE	admin	Felhasználó ID	200 OK	200 OK
api/admin/felhasznalok/{id}/aktiv	PATCH	admin	Felhasználó ID	200 OK	200 OK
api/admin/edzok	GET	admin	-	200 OK + lista	200 OK + lista
api/admin/edzok/{id}	DELETE	admin	Edző ID + újEdzoId	200 OK	200 OK
api/admin/edzok/{id}/minden-tagot-lat	PATCH	admin	Edző ID	200 OK	200 OK

api/admin/csapatok	GET	admin	-	200 OK + lista	200 OK + lista
api/admin/csapatok	POST	admin	Csapat adatai	200 OK	200 OK
api/admin/csapatok/{id}/edzo	PATCH	admin	EdzoId	200 OK	200 OK
api/admin/versenyek	GET	admin	-	200 OK + lista	200 OK + lista
api/admin/versenyek	POST	admin	Verseny adatai	200 OK	200 OK
api/admin/versenyek/{id}	PUT	admin	Módosított adatok	200 OK	200 OK
api/admin/versenyek/{id}	DELETE	admin	Verseny ID	200 OK	200 OK
api/Eredmenyek	GET	-	-	200 OK + lista	200 OK + lista
api/coach/tagok	GET	— (token nélkül)	-	401 Unauthorized	401 Unauthorized
api/admin/dashboard	GET	edzo (nem admin)	-	403 Forbidden	403 Forbidden

1.9 Reszponzívitás tesztek

A reszponzivitási tesztek célja annak ellenőrzése, hogy a Musztáng SE webalkalmázás megfelelően jelenik-e meg és működik-e különböző képernyőméreteken és eszközökön. A tesztelés kiterjedt az asztali számítógépekre, laptopokra, tabletekre és mobiltelefonokra egyaránt. Ellenőrzésre került a navigációs menü átalakulása mobilnézeten (hamburger menü), a tartalom elrendezésének változása különböző felbontásokon, valamint a gombok és űrlapok használhatósága érintőképernyőn. Minden tesztelt eszközön a felület hibátlanul alkalmazkodott a rendelkezésre álló képernyőterülethez.

Eszköz	Képernyőméret	Eredmény
MacBook Air 15”	2560 × 1664 px (15,3”)	Sikeres
iPhone 16 Pro	2622 × 1206 px (6,3”)	Sikeres
iPad (9. genráció)	2160 × 1620 px (10,2”)	Sikeres
Windows PC – 27” monitor	2560 × 1440 px (27”)	Sikeres